Sesión 24: Persistencia Local KV y SQL (Almacenamiento Rápido con SharedPreferences y Bases de Datos Relacionales Locales con SQFlite)

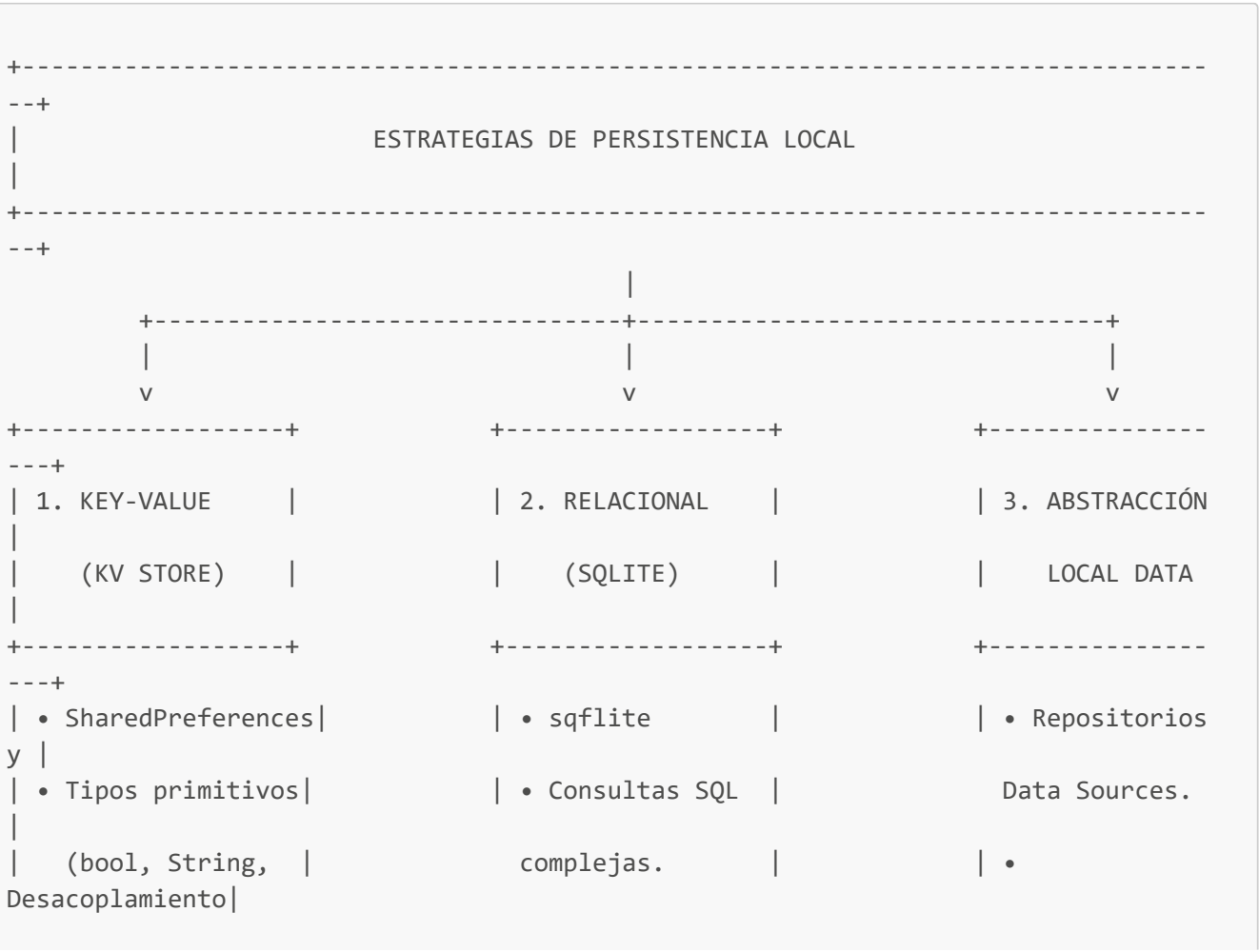
Introducción

La persistencia de datos local es un pilar fundamental en el desarrollo móvil para garantizar el funcionamiento sin conexión (*offline-first*), preservar las preferencias del usuario entre sesiones y almacenar estructuras complejas con alto rendimiento. La elección del mecanismo de almacenamiento adecuado depende de la naturaleza de los datos: configuraciones livianas de clave-valor o conjuntos de datos estructurados con relaciones complejas.

En esta sesión abordaremos las dos estrategias de persistencia más utilizadas en Flutter: almacenamiento **Key-Value (KV)** mediante `SharedPreferences` y almacenamiento relacional **SQL** mediante `sqflite`. Estudiaremos sus casos de uso, ciclo de vida, migraciones de esquema y las mejores prácticas para abstraerlos dentro de la capa de datos (*Data Layer*).

Conceptos y Definiciones

El almacenamiento persistente en Flutter se categoriza según la complejidad de la estructura de datos a almacenar.



```
|   int, double).   |           | • Transacciones |              de la UI.
|
| • Preferencias,   |             y migraciones.   |            | • Conversión
de  |
|   tokens, flags.  |           | • Tablas y PKs.  |              Map/DTO.
|
+------------------+            +------------------+            +---------------
---+
```

1. Almacenamiento Clave-Valor (`SharedPreferences`)

- **Propósito:** Guardar pequeños fragmentos de datos persistentes de forma asíncrona utilizando las plataformas nativas (`NSUserDefaults` en iOS y `SharedPreferences` en Android).
- **Tipos de Datos Soportados:** `int`, `double`, `bool`, `String` y `List<String>`.
- **Casos de Uso Ideales:** Banderas de configuración (*flags* de primera apertura), temas gráficos (modo oscuro/claro), identificadores de sesión o configuraciones de usuario.
- **Limitaciones:** No está diseñado para almacenar grandes volúmenes de datos ni estructuras anidadas complejas; las lecturas cargan el archivo en memoria.

---

2. Base de Datos Relacional (`sqflite`)

- **Propósito:** Plugin de SQLite para Flutter que permite ejecutar sentencias SQL completas, transacciones y consultas complejas en una base de datos local embebida.
- **Componentes Clave:**
- `openDatabase()`: Abre o crea el archivo SQLite, gestionando callbacks de ciclo de vida como `onCreate`, `onUpgrade` y `onConfigure`.
- `execute()`: Ejecuta instrucciones DDL/DML que no retornan conjuntos de datos (ej. `CREATE TABLE`).
- `insert()`, `query()`, `update()`, `delete()`: Métodos helper tipados que operan con diccionarios `Map<String, dynamic>`.
- `rawQuery()` / `rawInsert()`: Ejecución directa de código SQL arbitrario.
- **Control de Versiones y Migraciones:** La propiedad `version` dentro de `openDatabase` permite ejecutar scripts de actualización en `onUpgrade` sin perder datos de los usuarios.

Ejemplo Completo con Código Fuente: Sistema Integrado de Preferencias y Registro de Alumnos

El siguiente ejemplo implementa una aplicación completa que combina `SharedPreferences` para controlar la última fecha de sincronización y `sqflite` para gestionar un CRUD local de alumnos en ESCOM.

> **Nota de dependencia:** Para ejecutar este ejemplo se requieren los paquetes `sqflite: ^2.3.3+1`, `path: ^1.9.0` y `shared_preferences: ^2.2.3` en el archivo `pubspec.yaml`.

Código Fuente Explicado (`lib/main.dart`)

```dart
import 'dart:async';
import 'package:flutter/material.dart';
import 'package:path/path.dart' as p;
import 'package:shared_preferences/shared_preferences.dart';
import 'package:sqflite/sqflite.dart';

void main() {
  WidgetsFlutterBinding.ensureInitialized();
  runApp(const DemoPersistenciaApp());
}

// ============================================================================
// MODELO DE DATOS
// ============================================================================

class AlumnoLocal {
  final int? id;
  final String boleta;
  final String nombre;
  final double promedio;

  const AlumnoLocal({
    this.id,
    required this.boleta,
    required this.nombre,
    required this.promedio,
  });

  Map<String, dynamic> toMap() {
    return {
      'id': id,
      'boleta': boleta,
      'nombre': nombre,
      'promedio': promedio,
    };
  }

  factory AlumnoLocal.fromMap(Map<String, dynamic> map) {
    return AlumnoLocal(
      id: map['id'] as int?,
      boleta: map['boleta'] as String,
      nombre: map['nombre'] as String,
      promedio: (map['promedio'] as num).toDouble(),
    );
  }
}

// ============================================================================
// GESTOR DE BASE DE DATOS SQFLITE (SQL LOCAL)
// ============================================================================
```

```dart
class DatabaseHelper {
  static final DatabaseHelper instance = DatabaseHelper._init();
  static Database? _database;

  DatabaseHelper._init();

  Future<Database> get database async {
    if (_database != null) return _database!;
    _database = await _initDB('escom_local.db');
    return _database!;
  }

  Future<Database> _initDB(String filePath) async {
    final dbPath = await getDatabasesPath();
    final path = p.join(dbPath, filePath);

    return await openDatabase(
      path,
      version: 1,
      onCreate: _createDB,
    );
  }

  Future<void> _createDB(Database db, int version) async {
    await db.execute('''
      CREATE TABLE alumnos (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        boleta TEXT NOT NULL UNIQUE,
        nombre TEXT NOT NULL,
        promedio REAL NOT NULL
      )
    ''');
  }

  Future<int> insertarAlumno(AlumnoLocal alumno) async {
    final db = await instance.database;
    return await db.insert(
      'alumnos',
      alumno.toMap(),
      conflictAlgorithm: ConflictAlgorithm.replace,
    );
  }

  Future<List<AlumnoLocal>> obtenerTodosAlumnos() async {
    final db = await instance.database;
    final result = await db.query('alumnos', orderBy: 'id DESC');
    return result.map((json) => AlumnoLocal.fromMap(json)).toList();
  }

  Future<int> eliminarAlumno(int id) async {
    final db = await instance.database;
    return await db.delete(
      'alumnos',
```

```
      where: 'id = ?',
      whereArgs: [id],
    );
  }

  Future<void> cerrarDB() async {
    final db = await instance.database;
    db.close();
  }
}

// ============================================================================
// INTERFAZ DE USUARIO (PRESENTATION)
// ============================================================================

class DemoPersistenciaApp extends StatelessWidget {
  const DemoPersistenciaApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'ESCOM - Persistencia Local KV & SQL',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        useMaterial3: true,
        colorScheme: ColorScheme.fromSeed(
          seedColor: const Color(0xFF6B112D), // Guinda Institucional
        ),
      ),
      home: const PantallaPersistencia(),
    );
  }
}

class PantallaPersistencia extends StatefulWidget {
  const PantallaPersistencia({super.key});

  @override
  State<PantallaPersistencia> createState() => _PantallaPersistenciaState();
}

class _PantallaPersistenciaState extends State<PantallaPersistencia> {
  final _boletaController = TextEditingController();
  final _nombreController = TextEditingController();
  final _promedioController = TextEditingController();

  List<AlumnoLocal> _alumnos = [];
  String _ultimaSincronizacion = 'Nunca';
  bool _cargando = true;

  @override
  void initState() {
    super.initState();
    _cargarDatosLocales();
```

```
  }

  Future<void> _cargarDatosLocales() async {
    setState(() => _cargando = true);

    // 1. Cargar datos de SharedPreferences (Key-Value)
    final prefs = await SharedPreferences.getInstance();
    final fechaKV = prefs.getString('ultima_sincronizacion') ?? 'Sin registros';

    // 2. Cargar datos de SQFlite (SQL Relacional)
    final alumnosSQL = await DatabaseHelper.instance.obtenerTodosAlumnos();

    setState(() {
      _ultimaSincronizacion = fechaKV;
      _alumnos = alumnosSQL;
      _cargando = false;
    });
  }

  Future<void> _guardarAlumno() async {
    if (_boletaController.text.isEmpty ||
        _nombreController.text.isEmpty ||
        _promedioController.text.isEmpty) {
      return;
    }

    final nuevoAlumno = AlumnoLocal(
      boleta: _boletaController.text,
      nombre: _nombreController.text,
      promedio: double.tryParse(_promedioController.text) ?? 0.0,
    );

    // Guardar en SQL
    await DatabaseHelper.instance.insertarAlumno(nuevoAlumno);

    // Actualizar marca de tiempo en SharedPreferences
    final prefs = await SharedPreferences.getInstance();
    final ahora = DateTime.now().toString().split('.')[0];
    await prefs.setString('ultima_sincronizacion', ahora);

    _boletaController.clear();
    _nombreController.clear();
    _promedioController.clear();

    await _cargarDatosLocales();
  }

  Future<void> _eliminarAlumno(int id) async {
    await DatabaseHelper.instance.eliminarAlumno(id);
    await _cargarDatosLocales();
  }

  @override
  Widget build(BuildContext context) {
```

```
    return Scaffold(
      appBar: AppBar(
        title: const Text('Persistencia Local (KV & SQL)'),
        backgroundColor: Theme.of(context).colorScheme.primary,
        foregroundColor: Colors.white,
      ),
      body: _cargando
          ? const Center(child: CircularProgressIndicator())
          : SingleChildScrollView(
              padding: const EdgeInsets.all(16.0),
              child: Column(
                crossAxisAlignment: CrossAxisAlignment.start,
                children: [
                  // BANNER DE PREFERENCIAS KV (SharedPreferences)
                  Card(
                    color: Theme.of(context).colorScheme.primaryContainer,
                    child: ListTile(
                      leading: const Icon(Icons.access_time_filled),
                      title: const Text(
                        'Última Modificación (SharedPreferences KV):',
                        style: TextStyle(fontWeight: FontWeight.bold, fontSize:
13),
                      ),
                      subtitle: Text(_ultimaSincronizacion),
                    ),
                  ),
                  const SizedBox(height: 16),

                  // FORMULARIO DE REGISTRO
                  Text(
                    'Registrar en Base de Datos SQL (SQFlite)',
                    style: Theme.of(context).textTheme.titleMedium?.copyWith(
                          fontWeight: FontWeight.bold,
                        ),
                  ),
                  const SizedBox(height: 10),
                  TextField(
                    controller: _boletaController,
                    decoration: const InputDecoration(
                      labelText: 'Boleta',
                      border: OutlineInputBorder(),
                    ),
                  ),
                  const SizedBox(height: 8),
                  TextField(
                    controller: _nombreController,
                    decoration: const InputDecoration(
                      labelText: 'Nombre Completo',
                      border: OutlineInputBorder(),
                    ),
                  ),
                  const SizedBox(height: 8),
                  TextField(
                    controller: _promedioController,
```

```
                    keyboardType: TextInputType.number,
                    decoration: const InputDecoration(
                      labelText: 'Promedio General',
                      border: OutlineInputBorder(),
                    ),
                  ),
                  const SizedBox(height: 12),
                  SizedBox(
                    width: double.infinity,
                    child: ElevatedButton.icon(
                      onPressed: _guardarAlumno,
                      icon: const Icon(Icons.save),
                      label: const Text('Guardar Registro Local'),
                    ),
                  ),
                  const SizedBox(height: 20),

                  // LISTADO DE REGISTROS SQFLITE
                  Text(
                    'Alumnos Almacenados (${_alumnos.length})',
                    style: Theme.of(context).textTheme.titleMedium?.copyWith(
                          fontWeight: FontWeight.bold,
                        ),
                  ),
                  const SizedBox(height: 8),
                  _alumnos.isEmpty
                      ? const Center(
                          child: Padding(
                            padding: EdgeInsets.all(20.0),
                            child: Text('No hay registros en la base de datos
SQL.'),

                          ),
                        )
                      : ListView.builder(
                          shrinkWrap: true,
                          physics: const NeverScrollableScrollPhysics(),
                          itemCount: _alumnos.length,
                          itemBuilder: (context, index) {
                            final alumno = _alumnos[index];
                            return Card(
                              margin: const EdgeInsets.only(bottom: 8),
                              child: ListTile(
                                leading: CircleAvatar(
                                  child: Text('${alumno.id}'),
                                ),
                                title: Text(alumno.nombre),
                                subtitle: Text(
                                    'Boleta: ${alumno.boleta} | Promedio:
${alumno.promedio}'),

                                trailing: IconButton(
                                  icon: const Icon(Icons.delete, color:
Colors.red),

                                  onPressed: () => _eliminarAlumno(alumno.id!),
                                ),
```

```
                                ),
                              );
                            },
                          ),
                      ],
                    ),
                  ),
              );
          }
      }
```

Prácticas Guiadas / Laboratorio

Práctica 1: Migración de Esquema en SQFlite (`onUpgrade`)

1. Incrementa la versión de la base de datos a `2` dentro de la llamada `openDatabase`.
2. Implementa la función callback `onUpgrade(Database db, int oldVersion, int newVersion)` para agregar una nueva columna llamada `semestre` de tipo `INTEGER` a la tabla `alumnos` mediante una sentencia `ALTER TABLE`.
3. Actualiza el modelo `AlumnoLocal` y las consultas de inserción para soportar el nuevo campo preservando los datos preexistentes.

Práctica 2: Implementación de Transacciones para Inserción Masiva

1. Añade un método `Future<void> insertarAlumnosEnLote(List<AlumnoLocal> alumnos)` a la clase `DatabaseHelper`.
2. Utiliza `db.transaction((txn) async { ... })` e implementa un `Batch` (`txn.batch()`) para ejecutar múltiples inserciones en una única operación atómica.
3. Evalúa el rendimiento comparando el tiempo de ejecución de inserciones individuales versus el uso de transacciones en lote.

Resumen y Puntos Clave

- **Criterios de Selección de Persistencia:** Utiliza `SharedPreferences` para configuraciones simples y escalares (KV); emplea `sqflite` para estructuras complejas, consultas relacionales o colecciones masivas de datos.
- **Manejo de Asincronía:** Todas las operaciones en ambas soluciones retornan instancias `Future`, exigiendo un control riguroso de estados asíncronos en la UI.
- **Patrón Singleton para SQFlite:** Mantiene una única instancia abierta de la base de datos durante el ciclo de vida de la aplicación para evitar sobrecostos de apertura y bloqueos de archivo.
- **Mapeo DTO/Entity:** Mantén aislada la lógica de serialización mediante métodos `toMap()` y `fromMap()` para transformar registros de base de datos a objetos puros de Dart.

Bibliografía y Referencias

1. **Alex Tekartik.** (2024). *sqflite: SQLite plugin for Flutter*. Recuperado de: https://pub.dev/packages/sqflite
2. **Google LLC.** (2024). *Store key-value data on disk & Persist data with SQLite*. Recuperado de: https://docs.flutter.dev/cookbook/persistence
3. **Moedano, B.** (2023). *Desarrollo de Aplicaciones Móviles Adaptativas con Flutter*. Editorial Alfaomega. ISBN: 978-6075389102.